

Compiler Design

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

- (1) **Principles of Compiler Design** by Aho, Ullman

Please refer to the above books for more details.

Contents

1	Compiler Fundamentals	6
1.1	Introduction to Compilers	6
1.1.1	Major Parts of a Compiler	6
1.1.2	Compiler Architecture Diagram	7
1.2	Phases and Passes	7
1.2.1	Phases	7
1.2.2	Cross Compiler	7
2	Lexical Analysis	8
2.1	Overview	8
2.1.1	Tokens and Attributes	8
2.1.2	Process Flow	8
3	Syntax Analysis	9
3.1	Introduction	9
3.1.1	Key Concepts	9
3.2	Parser Types	9
3.2.1	Top-Down Parsers	9
3.2.2	Bottom-Up Parsers	9
3.3	Ambiguity in Grammars	9
3.3.1	Example of Disambiguation	10
4	Parsing Techniques	11
4.1	Top-Down Parsing	11
4.1.1	Recursive Descent Parsing	11
4.1.2	Predictive Parsing	11
4.2	Predictive Parser	11
4.2.1	Types of Predictive Parsing	11
4.3	Left Recursion	11
4.3.1	Left Factoring	12
4.4	YACC (Yet Another Compiler Compiler)	12
5	LL(1) Parsing	13
5.1	LL(1) Grammars	13
5.1.1	Properties of LL(1) Grammars	13
5.2	Constructing LL(1) Parsing Tables	13
5.2.1	Algorithm for Constructing LL(1) Parsing Table	13
5.2.2	Computing FIRST for Any String X	14
5.2.3	Computing FOLLOW (for non-terminals)	14

5.3	Example: LL(1) Parsing Table Construction	14
5.3.1	LL(1) Parsing Table	15
6	Bottom-Up Parsing	16
6.1	Introduction	16
6.1.1	Handle	16
6.2	Shift-Reduce Parsing Conflicts	16
6.2.1	Shift/Reduce Conflict	16
6.2.2	Reduce/Reduce Conflict	16
7	LR Parsing	17
7.1	LR Grammar Types	17
7.1.1	Grammar Hierarchy	17
7.2	LR Parsing	17
7.2.1	Unambiguous Grammars	17
8	Operator Precedence Parsing	18
8.1	Introduction	18
8.1.1	Operator Grammar Properties	18
8.2	Using Precedence Relations to Find Handles	18
8.2.1	Precedence Relation Rules	18
8.2.2	Precedence Table Example	18
8.3	LR Parsing Example	18
8.3.1	Advantages	19
8.3.2	Disadvantages	19
9	Intermediate Code Generation	20
9.1	Overview	20
9.1.1	Three Address Code	20
9.2	Representation Forms	20
9.2.1	Quadruples	20
9.2.2	Triples	20
9.2.3	Indirect Triples	21
10	Code Optimization	22
10.1	Common Sub-expression Elimination	22
10.2	Compile Time Evaluation	22
10.3	Variable Propagation	22
10.4	Dead Code Elimination	23
10.5	Code Motion	23
10.6	Induction Variables and Strength Reduction	23
11	Programming Language Principles	24
11.1	Typed versus Untyped Languages	24
11.1.1	Typed Languages	24
11.1.2	Weak and Strong Typing	24
11.2	Static versus Dynamic Typing	24
11.2.1	Static Typing	24
11.3	Storage Management	24

11.3.1 Activation Records	24
11.3.2 Types of Storage Management	25

Chapter 1

Compiler Fundamentals

1.1 Introduction to Compilers

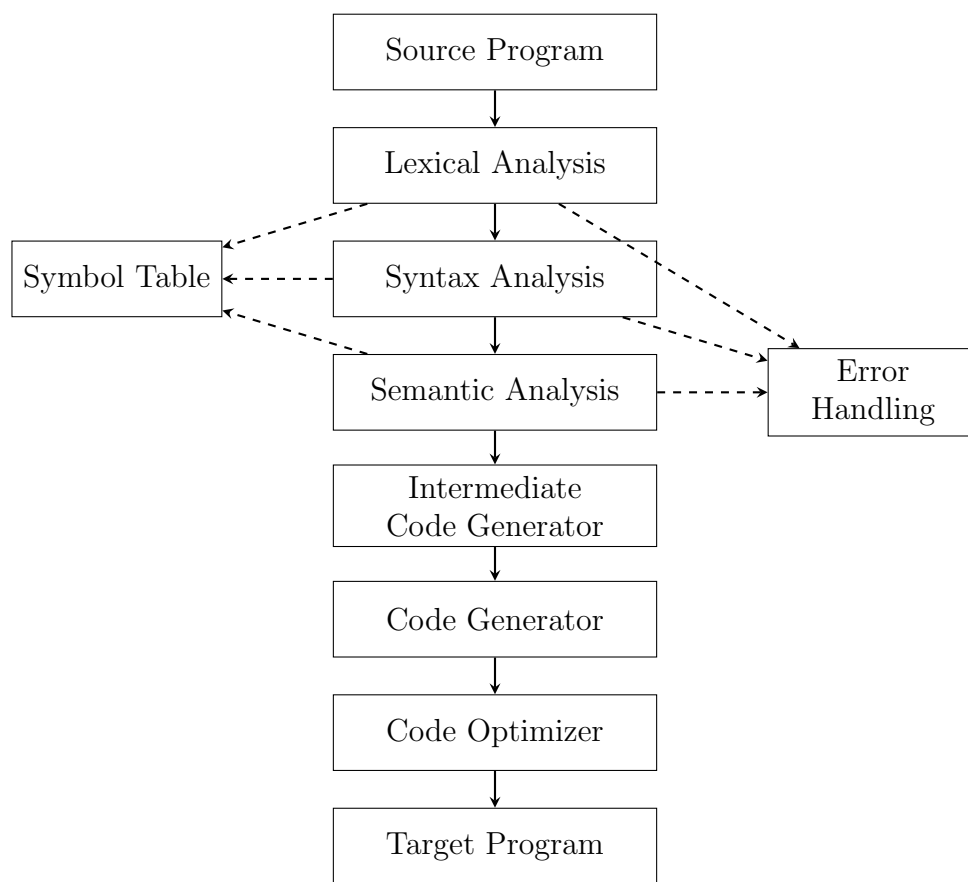
A compiler is a program that translates a high-level language program into a low-level language (machine code). The process involves multiple phases organized into two major groups.

1.1.1 Major Parts of a Compiler

The compilation process consists of two major phases:

- (1) **Analysis Phase:** An intermediate representation is created from the given source program
 - Lexical Analyzer
 - Syntax Analyzer
 - Semantic Analyzer (three phases in total)
- (2) **Synthesis Phase:** An equivalent target program is created from this intermediate representation
 - Intermediate Code Generator
 - Code Generator
 - Code Optimizer (three phases in total)

1.1.2 Compiler Architecture Diagram



1.2 Phases and Passes

1.2.1 Phases

Phases refer to the number of times the compiler has to traverse through the entire program. Several phases are grouped into one pass.

1.2.2 Cross Compiler

A cross compiler is a compiler that runs on one machine (A) and produces code for another machine (B).

Chapter 2

Lexical Analysis

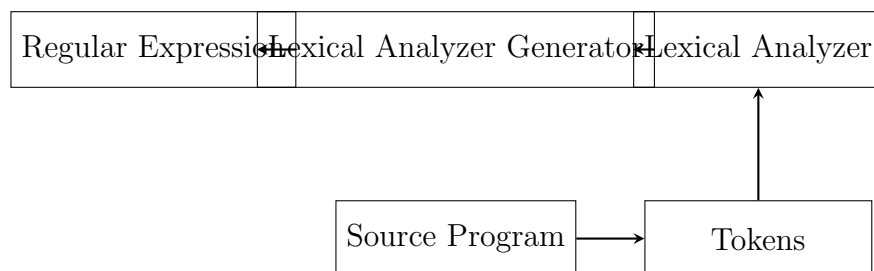
2.1 Overview

The Lexical Analyzer reads a source program character by character to produce tokens.

2.1.1 Tokens and Attributes

- A token can represent more than one lexeme; additional information should be held for that specific lexeme. This additional information is called the **attribute of the token**.
- Token type and its attribute uniquely identify a lexeme.
- Regular expressions are used to specify patterns.
- Finite Automata are used to recognize tokens.

2.1.2 Process Flow



Chapter 3

Syntax Analysis

3.1 Introduction

The Syntax Analyzer is also known as a parser. The syntax of a programming language is described by a Context-Free Grammar (CFG).

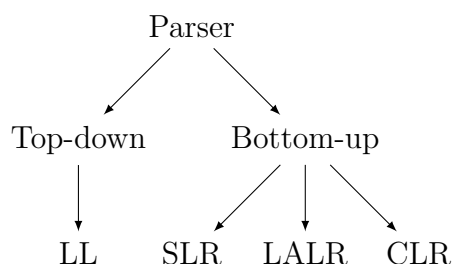
3.1.1 Key Concepts

- A CFG gives a precise syntactic specification of a programming language.
- The parser works on a stream of tokens.

3.2 Parser Types

3.2.1 Top-Down Parsers

Top-down parsers try to find the left-most derivation of the given source program.



Top-down parsers try to find the left-most derivation of the given source program.

Bottom-up parsers try to find the right-most derivation of the given source program in reverse order.

3.2.2 Bottom-Up Parsers

Both top-down and bottom-up parsers scan the input from left to right.

3.3 Ambiguity in Grammars

An unambiguous grammar should be written to eliminate ambiguity.

3.3.1 Example of Disambiguation

Consider the ambiguous grammar:

$$E \rightarrow E + E \mid E * E \mid E \wedge E \mid id \mid (E)$$

where $\wedge$ represents exponentiation (right associative) and $+$ represents addition (left associative).

Solution: To eliminate ambiguity, we rewrite the grammar as:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow G \wedge F \mid G$$

$$G \rightarrow id \mid (E)$$

Chapter 4

Parsing Techniques

4.1 Top-Down Parsing

4.1.1 Recursive Descent Parsing

- Backtracking is needed, which is not efficient
- Not used for general purpose parsing

4.1.2 Predictive Parsing

- No backtracking is required, making it efficient
- Uses LL(1) grammars

4.2 Predictive Parser

A predictive parser can uniquely choose a production rule by just looking at the current symbol.

The Non-Recursive (Table Driven) Predictive parser is also known as the LL(1) parser. We eliminate the left recursion in the grammar, and left-factor it. However, it may not be suitable for predictive parsing (i.e., not an LL(1) grammar).

4.2.1 Types of Predictive Parsing

- (1) Recursive Predictive Parsing
- (2) Non-Recursive Predictive Parsing (LL(1))

4.3 Left Recursion

Top-down techniques cannot handle left-recursive grammars.

4.3.1 Left Factoring

A predictive parser (a top-down parser without backtracking) insists that the grammar must be left-factored.

Example: Consider the grammar:

$$A \rightarrow a\beta B \mid a\beta \mid cdg \mid cdeB \mid cdf\beta$$

After left factoring:

$$\begin{aligned} A &\rightarrow aA' \mid cdg \mid cdeB \mid cdfB \\ A' &\rightarrow \beta B \mid \beta \end{aligned}$$

Further factoring:

$$\begin{aligned} A &\rightarrow aA'TcdA', \quad A' \rightarrow \beta B \mid B \\ A' &\rightarrow g \mid eB \mid fB \end{aligned}$$

4.4 YACC (Yet Another Compiler Compiler)

YACC generates C code for a syntax analyzer or parser.



Chapter 5

LL(1) Parsing

5.1 LL(1) Grammars

LL(1) stands for:

- First L: Input is scanned from left to right
- Second L: Left-most derivation is used as a lookahead symbol to determine parser action
- 1: One input symbol is used

5.1.1 Properties of LL(1) Grammars

- A grammar whose parsing table has no multiple entries is said to be an LL(1) grammar.
- A left-recursive grammar cannot be an LL(1) grammar.
- A grammar that is not left-factored cannot be an LL(1) grammar.
- An ambiguous grammar cannot be an LL(1) grammar.

5.2 Constructing LL(1) Parsing Tables

5.2.1 Algorithm for Constructing LL(1) Parsing Table

For each production rule $A \rightarrow \alpha$ of a grammar G :

- (1) For each terminal a in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$
- (2) If Λ (epsilon) is in $\text{FIRST}(\alpha)$, then:
 - For each terminal a in $\text{FOLLOW}(A)$, add $A \rightarrow \alpha$ to $M[A, a]$
 - If Λ is in $\text{FIRST}(\alpha)$ and $\$$ is in $\text{FOLLOW}(A)$, then add $A \rightarrow \alpha$ to $M[A, \$]$

All other undefined entries of the parsing table are error entries.

5.2.2 Computing FIRST for Any String X

- If X is a terminal symbol: $\text{FIRST}(X) = \{X\}$
- If X is a non-terminal symbol and $X \rightarrow A$ is a production rule, then A is in $\text{FIRST}(X)$
- If X is a non-terminal symbol and $X \rightarrow Y_1 Y_2 \dots Y_n$ is a production rule, then:
 - (i) If a terminal symbol a is in $\text{FIRST}(Y_1)$ and Λ is in all $\text{FIRST}(Y_i)$ for $i = 1, 2, \dots, j-1$, then a is in $\text{FIRST}(X)$
 - (ii) If Λ is in all $\text{FIRST}(Y_i)$ for $i = 1, \dots, n$, then Λ is in $\text{FIRST}(X)$

5.2.3 Computing FOLLOW (for non-terminals)

- If S is the start symbol, then $\$$ is in $\text{FOLLOW}(S)$
- If $A \rightarrow \alpha B \beta$ is a production rule, then everything in $\text{FIRST}(\beta)$ is in $\text{FOLLOW}(B)$ except Λ
- If $(A \rightarrow \alpha B)$ or $(A \rightarrow \alpha B \beta \text{ and } \Lambda \text{ is in } \text{FIRST}(\beta))$, then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$

5.3 Example: LL(1) Parsing Table Construction

Consider the grammar:

$$\begin{aligned} E &\rightarrow TE', & E' &\rightarrow +TE' \mid \Lambda \\ T &\rightarrow FT', & T' &\rightarrow *FT' \mid \Lambda \\ F &\rightarrow (E) \mid id \end{aligned}$$

Solution:

First, compute FIRST sets:

$$\begin{aligned} \text{FIRST}(F) &= \{ (, id \} \\ \text{FIRST}(T) &= \{ *, \Lambda \}, & \text{FIRST}(T') &= \{ (, id \} \\ \text{FIRST}(E') &= \{ +, \Lambda \}, & \text{FIRST}(E) &= \{ (, id \} \\ \text{FIRST}(TE') &= \{ (, id \}, & \text{FIRST}(+TE') &= \{ + \} \\ \text{FIRST}(*FT') &= \{ * \}, & \text{FIRST}(FT') &= \{ (, id \} \\ \text{FIRST}((E)) &= \{ (\} \\ \text{FIRST}(id) &= \{ id \} \end{aligned}$$

Next, compute FOLLOW sets:

$$\begin{aligned} \text{FOLLOW}(F) &= \{ \$,) \}, & \text{FOLLOW}(E) &= \{ \$,) \} \\ \text{FOLLOW}(T) &= \{ +,), \$ \}, & \text{FOLLOW}(T') &= \{ +,), \$ \} \\ \text{FOLLOW}(E') &= \{ *, +,), \$ \} \end{aligned}$$

5.3.1 LL(1) Parsing Table

	id	+	*	(	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	
E'		$E' \rightarrow +TE'$			$E' \rightarrow \Lambda$
T	$T \rightarrow FT'$			$T \rightarrow FT'$	
T'		$T' \rightarrow \Lambda$	$T' \rightarrow *FT'$		$T' \rightarrow \Lambda$
F	$F \rightarrow id$			$F \rightarrow (E)$	

Chapter 6

Bottom-Up Parsing

6.1 Introduction

Bottom-up parsing is also known as shift-reduce parsing.

6.1.1 Handle

If the grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.

A right-most derivation in reverse can be obtained by handle-pruning.

6.2 Shift-Reduce Parsing Conflicts

6.2.1 Shift/Reduce Conflict

This conflict occurs when the parser cannot decide which of several reductions to make.

6.2.2 Reduce/Reduce Conflict

The parser cannot decide which of several reductions to make.

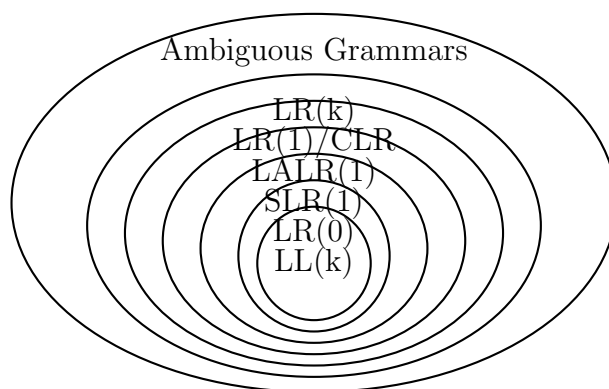
If a shift-reduce parser cannot be used for a grammar, that grammar is called a Non-LR(k) grammar.

Chapter 7

LR Parsing

7.1 LR Grammar Types

LR grammars represent a family of languages that can be parsed using LR techniques:



7.1.1 Grammar Hierarchy

$$LL(k) \subset LR(k)$$

$$LR(0) \subset SLR(1) \subset LALR(1) \subset LR(1)/CLR \subset LR(k)$$

Canonical LR is also referred to as LR(1).

7.2 LR Parsing

LR parsing is a non-backtracking shift-reduce parsing technique.

An LR parser can detect a syntactic error as soon as it is possible to do so in a left-to-right scan of the input.

7.2.1 Unambiguous Grammars

The diagram shows the relationship between different grammar classes, with unambiguous grammars forming nested subsets, and ambiguous grammars existing outside these structured families.

Chapter 8

Operator Precedence Parsing

8.1 Introduction

Operator precedence parsing is a small but important class of grammars.

8.1.1 Operator Grammar Properties

In an operator grammar, no production rule can have:

- (i) Λ at the right side
- (ii) Two adjacent or non-terminals at the right side

8.2 Using Precedence Relations to Find Handles

8.2.1 Precedence Relation Rules

- (1) Scan the string from the left end until the first $>$ is encountered
- (2) Then scan backwards (to the left) over any $=$ until a $<$ is encountered
- (3) The handle contains everything to the left of the first $>$ and to the right of the $<$ is encountered

8.2.2 Precedence Table Example

	id	+	*	\$
id		$>$	$>$	$>$
+	$<$	$>$	$<$	$>$
*	$<$	$>$	$>$	$>$
\$	$<$	$<$	$<$	

8.3 LR Parsing Example

Consider the grammar: $E \rightarrow E + E$, $E \rightarrow E * E$, $E \rightarrow id$

Stack	Input	Action
\$	$id + id * id$ \$	Shift
\$ id	$+id * id$ \$	Reduce
\$ E	$+id * id$ \$	Shift
\$ E +	$id * id$ \$	Shift
\$ E + id	$*id$ \$	Reduce
\$ E + E	$*id$ \$	Shift
\$ E + E *	id \$	Shift
\$ E + E * id	\$	Reduce
\$ E + E * E	\$	Reduce
\$ E + E	\$	Reduce
\$ E	\$	Accept

8.3.1 Advantages

Operator precedence parsing is powerful enough for expressions in programming languages.

8.3.2 Disadvantages

- (1) It cannot handle the unary minus (the lexical analyzer should handle this)
- (2) It is difficult to decide which language is recognized by the grammar

Chapter 9

Intermediate Code Generation

9.1 Overview

Intermediate codes are machine-independent codes that are close to machine instructions. Syntax trees, Postfix Notation, and Three-address codes can be used as intermediate languages.

9.1.1 Three Address Code

Three address code is represented as:

$$A = B * (C + D)$$

Three address code is as follows:

$$\begin{aligned}T_1 &= -B \\T_2 &= C + D \\T_3 &= T_1 * T_2 \\A &= T_3\end{aligned}$$

9.2 Representation Forms

9.2.1 Quadruples

	Operator	Operand1	Operand2	Result
(1)	−	B		T_1
(2)	+	C	D	T_2
(3)	*	T_1	T_2	T_3
(4)	=	T_3		A

9.2.2 Triples

	Operator	Operand1	Operand2
(1)	−	B	
(2)	+	C	D
(3)	*	(1)	(2)
(4)	=	A	(3)

9.2.3 Indirect Triples

Statement			Operator	Operand1	Operand2
(0)	(56)	(56)	−	B	
(1)	(57)	(57)	+	C	D
(2)	(58)	(58)	*	(56)	(57)
(3)	(59)	(59)	=	A	(58)

Chapter 10

Code Optimization

10.1 Common Sub-expression Elimination

If an expression has already been computed and the values of variables in the expression have not changed, we can use the previously computed value.

Example:

$$\begin{aligned}a &= (b * c) \quad \Rightarrow \quad a = T_1 \\T_1 &= b * c \\d &= (b * c) + x - y \quad \Rightarrow \quad d = T_1 + x - y\end{aligned}$$

10.2 Compile Time Evaluation

If all operands of an expression are constants, the expression can be evaluated at compile time.

Example:

$$A = 2 * (22.0/7.0) * r$$

Perform $2 * (22.0/7.0)$ at compile time.

Example 2:

$$\begin{aligned}x &= 12.4 \quad \Rightarrow \quad \text{Evaluate } x/2.3 \text{ at compile time} \\y &= x/2.3\end{aligned}$$

10.3 Variable Propagation

Example:

$$\begin{array}{ll}c = a * b & c = a * b \\x = a & \Rightarrow x = a \\d = a * b + 4 & d = a * b + 4\end{array}$$

After variable propagation, $a * b$ and $x * b$ will be identified as common subexpressions.

10.4 Dead Code Elimination

Variable propagation often leads to making assignment statements into dead code.

Example:

$c = a * b$ $x = a$ $d = a * b + 4$	$\Rightarrow$	$c = a * b$ $d = a * b + 4$
---	---------------	------------------------------------

10.5 Code Motion

This technique reduces the evaluation frequency of expressions by bringing loop-invariant statements out of the loop.

Example:

$a = 200;$ while ($a > 0$) $b = x + y;$ { while ($a > 0$) $b = x + y;$ $\Rightarrow$ { if ($a \times b == 0$) print("x d", 0); } }	$a = 200;$ if ($a \times b == 0$) print("x d", 0);
--	--

10.6 Induction Variables and Strength Reduction

An induction variable is used in a loop for the following kind of assignment:

$$i = i + \text{constant}$$

Strength reduction means replacing the high-strength operator by a low-strength operator.

Example:

$i = 1;$ while ($i < 10$) $j = 4;$ { while ($i < 40$) $y = j * 4;$ $\Rightarrow$ { } $y = 1;$ $i = i + 4;$ } }	$i = 1;$
---	----------

Chapter 11

Programming Language Principles

11.1 Typed versus Untyped Languages

11.1.1 Typed Languages

A language is typed if the specification of every operation defines the types of data to which the operation is applicable.

In contrast, an untyped language (such as most assembly languages) allows any operation to be performed on any data, which are generally considered to be sequences of bits of various lengths.

11.1.2 Weak and Strong Typing

- **Weak typing** allows a value of one type to be treated as another.
- **Strong typing** prevents the above situation.

Strongly typed languages are often termed as type-safe.

11.2 Static versus Dynamic Typing

11.2.1 Static Typing

A programming language is said to be statically typed when type checking is performed during compile time.

A programming language is said to be dynamically typed when the majority of its type checking is performed at run-time.

In dynamically typed languages, values have types but variables do not; i.e., a variable can refer to a value of any type.

11.3 Storage Management

11.3.1 Activation Records

Information needed by a single execution of a procedure is managed using a contiguous block of storage called an activation record.

An activation record is allocated when a procedure is entered, and it is deallocated when that procedure exits.

11.3.2 Types of Storage Management

(1) **Static Storage Management:**

Static allocation is done during compilation that remains fixed throughout execution. It requires no run-time storage management software. Global data in C is allocated using static storage. It is very efficient, but it is incompatible with recursive subprogram calls.

(2) **Dynamic Storage Management:**

Also termed as heap storage management. The need for heap storage arises when a language permits storage to be allocated and freed at arbitrary points during program execution.

(3) **Stack-based Storage Management:**

It is used when storage requirements are not known at compile time, but the requests obey a Last In First Out order.

Examples:

- Local variables in a procedure in C
- Procedure call information (return address, etc.)

Stack-based allocation is used in recursive sub-programs.